# FOL: A Functional Object Lisp

Frank Adrian
Ancar Technology
Olathe, KS, USA
frank.adrian314159@gmail.com

## Abstract

FOL (Functional Object Lisp) synthesizes CLOS object orientation with persistent data structures, demonstrating that these paradigms are synergistic rather than competing. While Lisp allows for a wide variety of programming styles, the gap between the CLOS-centric imperative model and the Clojure-centric functional model remains significant. FOL bridges this by providing a CLOS implementation where every object is internally represented by a persistent data structure, allowing researchers and developers to leverage the best of both worlds: the extensibility of the MOP and the versioning power of persistent state. We present a design where persistent slot storage is enabled via a CLOS metaobject protocol (MOP) adaptation, utilizing a hybrid native-slot/overflow-trie layout ($T = 8$) to preserve $O(1)$ access for the common case. We further introduce a four-level category-based specificity ordering for predicate dispatch that provides a decidable, predictable alternative to logical subsumption. Our evaluation indicates that while functional updates incur significant micro-overhead (33–447×), this cost effectively amortizes in non-trivial workloads. In a 4,096-gate logic simulation, FOL reaches near-parity (1.27×) with native Common Lisp and, crucially, reduces memory pressure by 0.59× via structural sharing, suggesting that the "cost of persistence" becomes a modular advantage at scale.

## CCS Concepts

• **Software and its engineering** → **Functional languages**; **Object oriented languages**; *Extensible languages.*

## Keywords

Functional programming, object-oriented programming, Lisp, persistent data structures, CLOS, MOP, transducers

FOL explores the synthesis of Common Lisp-style object orientation (CLOS) and Clojure-style immutable data structures. We find that multiple dispatch over persistent state enables architectural patterns that are more natural than in either parent paradigm alone. While CLOS offers powerful method combinations and a metaobject protocol (MOP), its root in mutable state complicates versioned history and transactional integrity. Conversely, Clojure's immutable model lacks the introspective extensibility provided by the MOP, requiring externalized state management (atoms, refs) that sits outside the object hierarchy.

Immutable objects simplify concurrency by eliminating synchronization and manual locking. FOL adapts the CLOS MOP to use persistent data structures for slot storage. While Racket and records in ML-family languages offer similar guarantees, FOL's contribution is applying persistence to full CLOS with method combinations, `slot-missing` overflow, and lazy schema migration. We implemented FOL as a standalone Lisp to enforce persistence invariants that library-based approaches in Common Lisp cannot easily guarantee. Library approaches typically rely on discipline (avoiding `setf`) or macros that wrap slot access; by implementing a custom metaclass that intercepts the MOP's low-level slot accessors, we ensure that persistence is a property of the object itself, not the calling code.

The synthesis of immutability and CLOS makes several patterns more ergonomic:

- **Versioned diffs**: Using `:around` methods on `assoc` to detect and record field-level changes automatically across functional updates.
- **Hierarchical speculative execution**: Layering `:around` guards to validate updates through a chain of invariants, with automatic rollback by returning the pre-update snapshot.
- **Observable notifications**: Routing change events via predicate dispatch without manual `cond` cascades.

These patterns leverage the fact that every functional update in FOL is a first-class event that can be intercepted, validated, and transformed by the object's metaclass.

## 1 Language Design

### 1.1 Core Philosophy

FOL's design commitments are: (1) **Immutability by default** for all objects and collections; (2) **Structural sharing** to amortize update costs; (3) **Object identity** distinct from value equality; (4) **Generic functions** with destructuring; and (5) **MOP-integrated persistence**.

### 1.2 Identity, Syntax, and Persistence

We follow Driscoll et al. [10] in using *persistent* to mean preserving previous versions via structural sharing rather than database persistence. Following Hickey's epochal time model [15], FOL distinguishes between the *identity* of a value (which never changes) and the *identity* of an entity (which may progress through a sequence of values). Reference identity via `eq` refers to a specific version of an object; structural value equality via `=` compares the contents of two versions. Syntactically, FOL adopts Clojure's literal syntax: `[1 2 3]` for vectors, `{:a 1}` for maps, and `#{1 2}` for sets. User-defined classes, declared via `defclass`, inherit from `<persistent-object>` by default. Functional updates via `assoc` produce new instances while preserving the original, ensuring that "modification" is always a non-destructive transformation of the reference:

```
(defclass <person> [] [[name] [age]])
(def alice (make <person> :name "Alice" :age 30))
(def older-alice (assoc alice :age 31))
```

Slot storage uses a hybrid strategy: objects with $\leq 8$ slots use native CLOS slots for speed, while wider objects overflow into a persistent vector trie ($T=8$). This threshold keeps ≈80% of classes on the all-native fast path while preserving $O(1)$ access for small objects and $O(1) + O(\log n)$ for larger ones.

# 2 Architecture and Implementation

## 2.1 Metaobject Protocol Adaptation

MOP adaptation, detailed in Table 1, is the primary mechanism for enforcing persistence. Unlike library-based approaches that rely on caller discipline, FOL ensures immutability by intercepting the low-level slot access protocols. This is mandatory for three reasons: (1) transparent mutation-guarding demands metaclass-level interception to prevent `(setf slot-value)` calls; (2) the structural sharing model requires mapping logical slots to a hybrid physical layout (native vs. overflow); and (3) lazy schema evolution (Section 3.3) requires intercepting access to transform legacy versions on-the-fly.

The `persistent-class` metaclass reroutes access by specializing `slot-value-using-class` based on index. Small objects ($\leq 8$ slots) use native CLOS slots for speed, while wider objects overflow into a persistent trie. The `(setf slot-value-using-class)` method is specialized to block post-initialization writes, signaling a `mutation-error` on update attempts.

The hybrid layout uses a specialized `persistent-effective-slot-definition` class. During initialization, the metaclass calculates slot indices. If the index is $\leq$ 8, it allocates a standard CLOS slot; otherwise, it marks the slot as virtual. Virtual slot access via `slot-value-using-class` triggers a lookup in the overflow trie (`%persistent-vector`) using the slot name identity. Because HAMT lookups are $O(\log_{32} N)$, even a class with 1,000 slots maintains fast access. The metadata slot (`%metadata`) remains native, preventing infinite recursion in the introspection logic.

## 2.2 Collection Implementation

FOL implements nineteen persistent collection classes including **Vectors** (32-way branching persistent tries), **Maps** (HAMTs), **Sets** (HAMTs), **Sorted collections** (B-trees), and **Lists** (persistent cons cells). HAMT lookup is only 1.3× slower than CL `gethash` for the common case; however, while a hash table exhibits $O(1)$ average-case performance with occasional $O(N)$ rehash spikes, the HAMT provides $O(\log_{32} N)$ bounded performance—at most six probes for a billion-element map—trading amortized $O(1)$ for predictable worst-case latency. The B-tree implementation for sorted collections uses 32-way branching to minimize cache misses during traversal.

## 2.3 Runtime Representations

Most runtime representations utilize a Common Lisp base. Primitives, functions, packages, and error handling derive directly from CL. Garbage collection ensures that stale snapshots and shared trie nodes are reclaimed when unreferenced.

# 3 Features

## 3.1 Clojure Features

FOL implements key Clojure abstractions:

- **Transducers** [16]: Composable transformation pipelines that decouple transformation logic from the input source. FOL transducers are plain higher-order functions (closures) applied via `transduce`. Any collection type can participate as a source by implementing `collection-seq`; any collection type can serve as a sink by implementing `conj`. This generic-function-based approach is more extensible than Clojure's interface-based model, as it allows adding transducer support to existing third-party or native classes without internal code modification.
- **Lazy sequences**: On-demand computation using a `delay`-based memoization strategy. FOL's lazy sequences are *realized* only once and cached, ensuring that expensive computations are amortized across multiple traversals.
- **Atoms/Refs/Agents**: Concurrency primitives for managed mutable state. Atoms provide uncoordinated, synchronous change via compare-and-swap (CAS), while Refs provide coordinated, synchronous change via software transactional memory (STM). Agents provide independent, asynchronous state: actions dispatched via `send` are queued and executed serially on a dedicated per-agent background thread.
- **Sequence functions**: Over 100 polymorphic functions including `map`, `filter`, and `reduce`.
- **Macros**: Clojure-style `defmacro` with syntax-quote, unquote, and auto-gensym.

**Transient builders** provide a mutation-windowing protocol that reduces functional-update costs from $O(k)$ HAMT path copies to $O(1) + O(\log n)$ for batch writes. To ensure safety, FOL enforces *thread-confined runtime guards*: the `transient` call returns a shallow copy with a volatility token set to the *current thread ID*; further mutations (`assoc!`) verify this token and signal an error if the object is accessed from a different thread or has already been frozen via `persistent!`. FOL additionally provides `transfer-ownership!` for linear handoffs in actor-pipeline patterns, where Thread A builds a transient and passes it to Thread B for further enrichment before freezing. The call atomically replaces the owner token with the new thread's identity—simultaneously invalidating the original thread's access so that any subsequent `assoc!` or `persistent!` by the original thread signals an error. This is a runtime approximation of **uniqueness types** [2, 22]: it provides use-after-transfer detection rather than static prevention. By linking volatility to thread identity, FOL approximates the "destructive read" semantics of linear types without global static analysis. This pragmatic choice catches concurrency errors during development without the verification burden of a full linear type system.

## 3.2 Generic Function Integration

FOL's generic functions dispatch by arity first, then by type predicates and specificity within an arity group. Methods at different arities may coexist on the same generic:

```
(defgeneric process [x])
(defmethod process
  ([(x <vector>)] (vec (map process x)))
  ([(x <dict>)]   (update-vals x process))
  ([(x <number>)] (* x 2)))

(process [1 {:a 2 :b 3} [[4]]])  ; => [2 {:a 4 :b 6} [[8]]]
```

**Table 1: MOP Protocol Adaptation**

| Protocol | Status | Rationale |
|---|---|---|
| `slot-value-using-class` | Redirected | Reads from base object or overflow trie |
| `(setf slot-value-using-class)` | Guarded | Blocks post-init writes |
| `slot-boundp-using-class` | Redirected | Checks object slot membership |
| `slot-makunbound-using-class` | Blocked | Immutability invariant |
| `validate-superclass` | Extended | Cross-metaclass mixing |
| `initialize-instance` | Extended | Populates base object and overflow trie from initargs |
| `update-instance-for-redefined-class` | Omitted | Instances are immutable; lazy migration instead (Sec. 3.3) |
| `change-class` | Omitted | Violates immutability |
| *Class computation, method combination, allocation: standard CLOS behavior inherited unchanged.* | | |

Multi-clause `defmethod` forms expand to separate method definitions. The `:before`, `:after`, and `:around` qualifiers apply per-clause; `call-next-method` follows CLOS semantics.

**`defn` vs `defgeneric`/`defmethod`**: Every FOL function is dispatched by specificity. `defn` defines a "closed" multi-clause function where all clauses are colocated, similar to pattern-matching in Haskell or Clojure's `defn`. This is the idiomatic choice for fixed domain logic. The FOL compiler optimizes `defn` by emitting a single `defun` that performs `cond`-based routing over the provided patterns in specificity order. Because all clauses are known at compile-time, no dynamic method-set scan is required; SBCL then applies its standard native-code optimizations to the emitted `cond` form.

In contrast, `defgeneric` and `defmethod` provide "open" extensible dispatch. This extensibility requires a dynamic specificity scan on every invocation. Qualifiers like `before`, `after`, and `around` are restricted to `defmethod` to allow `defn` to remain a high-speed routing form.

Figure 1 gives the formal grammar for FOL's multi-pattern dispatch syntax.

Note that *pattern* forms defined in the grammar lack a body (used for `defgeneric` declarations), while *clause* forms combine a *pattern* and evaluating *body* (used for implementations).

FOL extends pattern matching with predicate specializers. The syntax `(var (fn args...))` evaluates `(fn var args...)` at runtime. Patterns are ordered by *constraint strength* across four syntactically recognizable levels (Table 2), using definition order as a tie-breaker. This *syntactic precedence model* avoids the undecidability of logical subsumption by using the observable form of the specializer as a proxy for narrowness. Lexicographic comparison of level vectors ensures left-to-right scanning dominance.

**Table 2: Predicate Specificity Categories**

| Level | Category | Example |
|---|---|---|
| 3 | Predicates | `(x (even?))` |
| 2 | Types | `(x <integer>)` |
| 1 | Destructuring | `[x y]` |
| 0 | Wildcard | `x` |

*3.2.1 Formal Specificity Ordering.* We formalize the specificity relation $\prec$ using the inference rules summarized in Table 3. Let $\mathcal{L}(p) \in \{0, 1, 2, 3\}$ be the specificity level of pattern $p$.

**Table 3: Specificity Inference Rules**

| Rule | Formalization |
|---|---|
| Spec-Level | $\dfrac{\mathcal{L}(p_1) > \mathcal{L}(p_2)}{p_1 \prec p_2}$ |
| Spec-Prefix | $\dfrac{p_1 \prec q_1}{[p_1, \dots] \prec [q_1, \dots]}$ |
| Spec-Tail | $\dfrac{\mathcal{L}(p_1) = \mathcal{L}(q_1) \qquad [p_2, \dots] \prec [q_2, \dots]}{[p_1, \dots, p_n] \prec [q_1, \dots, q_n]}$ |
| Spec-Length | $\dfrac{\mathcal{L}(p_i) = \mathcal{L}(q_i)\ \forall i \in \{1 \dots n\}}{[p_1, \dots, p_n, p_{n+1}] \prec [q_1, \dots, q_n]}$ |
| Spec-Fixed | $\dfrac{\mathcal{L}(p_i) = \mathcal{L}(q_i)\ \forall i \in \{1 \dots n\}}{[p_1, \dots, p_n] \prec [q_1, \dots, q_n\ \&\ r]}$ |
| Spec-Map | $\dfrac{p \prec q}{\{k\ p, \dots\} \prec \{k\ q, \dots\}}$ |
| Spec-Keys | $\dfrac{\text{keys}(q) \subset \text{keys}(p)}{p \prec q}$ |

The operational dispatch order $\prec_{\text{disp}}$ resolves ties using definition order. Let $idx(p)$ denote the definition index of the method clause containing $p$.

$$p_1 \prec_{\text{disp}} p_2 \iff p_1 \prec p_2 \vee (\neg(p_2 \prec p_1) \wedge idx(p_1) < idx(p_2))$$

Patterns of different arity are incomparable; dispatch selects the arity group first, then applies total order $\prec_{\text{disp}}$. Lexicographic levels ensure $p \prec_{\text{disp}} q$ or $q \prec_{\text{disp}} p$ for all same-arity patterns. While definition-order tie-breaking is a modularity hazard, it follows Lisp's interactive workflow.

Lemma 3.1 (Totality). *$\prec_{disp}$ is a strict total order on same-arity patterns.*

Proof sketch. Every pattern position $p_i$ is mapped to a level $\mathcal{L}(p_i) \in \{0, 1, 2, 3\}$. For sequential patterns, $\prec$ induces a lexicographic total order on level vectors; Spec-Length orders patterns

| | | | |
|---|---|---|---|
| *defn* | ::= | `(defn` *name* (*single-clause* \| *multi-clause*)) | |
| *defgeneric* | ::= | `(defgeneric` *name* (*single-pattern* \| *multi-pattern*)) | |
| *defmethod* | ::= | `(defmethod` *name qualifier*? (*single-clause* \| *multi-clause*)) | |
| *fn* | ::= | `(fn` (*single-clause* \| *multi-clause*)) | |
| *single-clause* | ::= | *pattern body*$^+$ | |
| *multi-clause* | ::= | (*pattern body*$^+$)$^+$ | |
| *single-pattern* | ::= | *pattern* | |
| *multi-pattern* | ::= | *pattern*$^+$ | |
| *pattern* | ::= | `[`*param*$^*$`]` \| `[`*param*$^*$ `&` *rest-param*`]` | |
| *param* | ::= | *symbol* | (simple binding) |
| | \| | (*symbol type*) | (type specializer) |
| | \| | (*symbol* (*fn-name arg*$^*$)) | (predicate specializer) |
| | \| | *destructure* | (nested destructuring) |
| *destructure* | ::= | `[`*param*$^*$`]` | (sequential) |
| | \| | `{:keys [`*key-spec*$^*$`]}` | (map) |
| *key-spec* | ::= | *symbol* \| (*symbol type*) \| (*symbol pred*) | |
| *type* | ::= | <*type-name*> | |
| *qualifier* | ::= | `:before` \| `:after` \| `:around` | |

**Figure 1: FOL Multi-Pattern Dispatch Grammar (simplified)**

of unequal length when all shared-position levels agree. For map patterns with the same key set, keys are enumerated in alphabetical order of their symbol names; Spec-Map is applied entry-by-entry lexicographically, and if all entries tie, *idx* resolves the comparison. Spec-Keys handles distinct key sets via inclusion. Importantly, sequential and map patterns of the same arity are structurally incomparable under the specificity relation $\prec$; in all such cross-form cases, the system "self-corrects" by falling back to the definition index *idx* to ensure a stable total order. Because *idx* is a strict total order on all patterns, every pair $p, q$ is resolved by $p \prec q$, $q \prec p$, or *idx*. We use definition-order tie-breaking rather than signaling ambiguity to align with Lisp's interactive, last-definition-wins workflow. □

Predicate dispatch requires an $O(N)$ linear scan per invocation as resolution cannot be statically cached. While this is conventionally seen as a performance drawback, it provides a predictable upper bound for real-time applications unlike theorem-prover-based dispatch which can trigger unpredictable latency. To mitigate this cost, FOL's generic function metaobjects partition methods by arity group. This ensures that a call to a 2-argument function only scans the subset of methods with matching arity, greatly reducing $N$ in most production workloads.

A planned optimization termed **dispatch stratification** would further reduce this scan. Methods would be partitioned into two strata: the *static stratum* (Levels 0–2) containing methods specialized only on types, wildcards, or structural destructuring; and the *dynamic stratum* (Level 3) containing methods with arbitrary predicates. Because Levels 0–2 are stable and decidable, the static stratum could be pre-compiled into a standard CLOS dispatch table or decision tree. When no Level 3 methods are registered for the arity group, the static stratum would be a complete fast path; otherwise both strata are scanned and the winner selected by $\prec_{\text{disp}}$, preserving full specificity semantics. Type predicates like `(<number>?)` are already recognized syntactically (Level 2) in anticipation of this optimization. While this syntactic boundary is a pragmatic compromise for decidability, it introduces a trade-off in referential transparency: aliasing a predicate symbol (e.g., via `def`) shifts its category to Level 0 (Wildcard) unless manually tagged. We accept this "semantic leaking" to preserve O(1) level calculation during method registration. Metadata-based markers (a planned extension; see Section 4.7) are the intended architectural resolution. `call-next-method` within specialized auxiliaries follows standard CLOS precedence, resolving the next most specific method according to the total order $\prec_{\text{disp}}$. Since methods operate on immutable snapshots, the effect of :around methods is inherently isolated, preventing the "unintentional side-effect" bugs common in mutable CLOS method chains.

*3.2.2 Emergent Programming Patterns.* FOL's persistence simplifies complex state patterns. **Structural Diff** (Listing 1) uses an `:around` method on `assoc` to detect changes by comparing snapshots, triggering automatic audit logs. **Speculative Execution** enables rejecting updates by returning the pre-update snapshot (`obj`), which is impossible in mutable CLOS where the next method modifies state in-place. **Content routing** via predicate dispatch (Listing 2) allows sensors to trigger alerts based on data values without manual `cond` cascades.

**Listing 1: Automatic Structural Diff**

```
(defmethod assoc :around [(obj <diffable>) key val]
  (bind [old (get obj key)
         new-obj (call-next-method)]
    (if (= old (get new-obj key))
        new-obj
        (assoc new-obj :_changes (inc (get obj :_changes 0))))))
```

**Listing 2: Value-Based Alerting**

```
(defmethod notify [({:keys [temp]} (>= 100))]
  (print "ALERT: Critical Temperature!"))
```

### 3.3 Schema Evolution

Standard CLOS redefinition assumes mutable instances. FOL instead uses *lazy schema evolution*: instances remain frozen Schema 1 snapshots, while functional updates (`assoc`) perform implicit migration by allocating Schema 2 instances and copying old slots forward. `:alias` annotations (Listing 3) allow renamed slots to pull legacy data during migration. Single-hop alias lookups are supported; multi-hop chains (aliasing an alias) are intentionally not resolved to prevent circular lookup dependencies and maintain $O(1)$ access latency. Unresolved chains signal a `slot-missing` error on access. Conflicting aliases are handled by standard initialization order.

**Listing 3: Lazy Schema Migration with Aliasing**

```
;; Schema 1
(defclass <sensor> [] [[id] [reading]])
;; Schema 2: rename reading -> val, keep history
(defclass <sensor> [] [[id] [val :alias reading]])
```

## 4 Evaluation

### 4.1 Comparison with Related Languages

**Table 4: Built-in Feature Comparison**

| Feature | FOL | Clojure | CL |
|---|---|---|---|
| Persistent objects | ✓ | ○[c] | × |
| CLOS/MOP | ✓ | × | ✓ |
| Method combinations | ✓ | × | ✓ |
| Predicate dispatch | ✓ | single[a] | ×[b] |
| Transducers | ✓ | ✓ | × |
| Lazy seqs | ✓ | ✓ | × |
| Macros | ✓ | ✓ | ✓ |

[a] `defmulti` provides single-parameter predicate dispatch (arbitrary fn) but lacks multi-parameter specificity ordering.
[b] Multi-parameter type dispatch natively; libraries like `trivia` add pattern matching.
[c] Clojure has persistent *data structures*; `defrecord` produces a map-backed value type, not a MOP-integrated persistent object.

FOL uniquely combines Clojure's functional features with Common Lisp's object system. Persistent objects with fewer than 8 slots maintain $O(1)$ access and mutable-equivalent update cost because they bypass the overflow trie entirely. Those with more than 8 slots and most collections incur $O(\log n)$ vs. $O(1)$ for mutable operations, but high branching factors ($n = 32$) keep constant factors small. The performance boundary of FOL is thus defined not by algorithm complexity, but by the constant-time multiplier of persistence.

FOL's transducer implementation sources any collection via `collection-seq` and sinks into any collection via `conj`. A new collection type integrates with the full transducer library by specializing these two generics, without modifying transducer code; this represents an architectural improvement over Clojure, which relies on hardcoded host-language interfaces that are difficult to extend to foreign or existing types.

**Listing 4: Naive Persistent Partition**

```
(defn partition [v low high]
  (bind [pivot (get v high)]
    (loop [j low i (dec low) curr-v v]
      (if (< j high)
          (if (<= (get curr-v j) pivot)
              (bind [next-i (inc i)
                     v1 (assoc curr-v next-i (get curr-v j))
                     v2 (assoc v1 j (get curr-v next-i))]
                (recur (inc j) next-i v2))
              (recur (inc j) i curr-v))
          (bind [next-i (inc i)
                 v1 (assoc curr-v next-i (get curr-v high))
                 v2 (assoc v1 high (get curr-v next-i))]
            [v2 next-i])))))
```

### 4.2 Benchmark Methodology

All measurements were taken on SBCL 2.6.0 (Windows 11, AMD Ryzen 9 5900X, 64 GB RAM). Feature benchmarks report the mean of 100 runs (coefficient of variation <2%); LSim benchmarks report the mean of 10 runs. For timing, we use `get-internal-run-time`; for allocation, we use SBCL's `(sb-ext:get-bytes-consed)`.

We compare against native Common Lisp rather than Clojure for several reasons. First, SBCL compiles to native code, providing a strict upper bound on overhead that isn't masked by JVM jit-warmup or garbage collector abstractions. Second, since FOL transpiles *to* Common Lisp, this isolates the cost of the language abstraction itself from the cost of the underlying runtime. Third, Clojure's object orientation is fundamentally different (protocol-based rather than class-based), whereas FOL's use of CLOS allows for a direct comparison of the "persistence tax" on the exact same object model.

### 4.3 Translation Penalty Analysis

Functional slot updates are 33–447× slower than in-place `(setf slot-value)` (quantified in Section 4.6), defining FOL's operating boundary for imperative workloads. FOL is structurally unsuitable for algorithmic workloads dominated by tight mutation loops *unless* the transient-builder protocol (Section 3.1) is used: transients batch multiple slot writes into a single allocation, reducing the cost of bulk updates to mutable-equivalent construction.

We evaluated naive persistent translations of quicksort and BFS to isolate the path-copying penalty; each `assoc` recreates the HAMT path to the root, generating intermediate nodes on every swap or visited vertex. Listing 4 shows the naive partition logic; Listing 5, the naive BFS step. Both implementations are correct but generate large numbers of intermediate HAMT nodes within their inner loops, creating significant GC pressure and runtime overhead.

Table 5 shows results for all three implementations on both workloads (SBCL 2.6.0, AMD Ryzen 9 5900X, 3 runs each).

For the 100,000-element quicksort, naive persistent FOL generates ~3.4 million HAMT path-copy allocations—two `assoc` calls per swap—producing 2,947 MB of garbage at 57.5× the CL time. For the 50,000-node BFS, naive persistent FOL incurs one `assoc` per visited node, generating 53 MB at 28.3×. The transient-builder protocol substantially recovers performance in both cases: transient quicksort reduces overhead to 20.8× with 97% less allocation (91 MB);

**Listing 5: Naive Persistent BFS Step**

```
(defn bfs-step [q dists graph]
  (if (empty? q)
      dists
      (bind [u (first q)
             edges (get graph u)
             d (get dists u)]
        (loop [es edges nq (rest q) nd dists]
          (if (empty? es)
              (bfs-step nq nd graph)
              (bind [v (first es)]
                (if (= (get nd v) -1)
                    (recur (rest es)
                           (conj nq v)
                           (assoc nd v (inc d)))
                    (recur (rest es) nq nd))))))))
```

**Table 5: Adversarial Workload Results**

| Workload | Implementation | Time | Alloc | vs. CL |
|---|---|---|---|---|
| QuickSort | CL (simple-vector) | 23 ms | 0.8 MB | 1.0× |
| $N = 100{,}000$ | FOL Persistent (assoc) | 1310 ms | 2947 MB | 57.5× |
| | FOL Transient (assoc!) | 474 ms | 91 MB | 20.8× |
| BFS | CL (hash-table) | 2 ms | 2.5 MB | 1.0× |
| $N = 50{,}000$ | FOL Persistent (assoc) | 52 ms | 53 MB | 28.3× |
| | FOL Transient (assoc!) | 11 ms | 8.2 MB | 5.8× |

transient BFS reduces overhead to 5.8× with 85% less allocation (8.2 MB), as `hamt-assoc!` mutates the distance map in place across the traversal before freezing the result with `hamt-persistent!`. These results establish a **translation penalty** for imperative algorithms; they do not define the language's expressivity boundary; idiomatic persistent algorithms (e.g., merge sort over persistent vectors, BFS with a persistent queue) avoid per-element path copies and would produce substantially lower allocation profiles, though their empirical measurement is left to future work.

The stark difference between naive persistence and transient-based updates highlights that the primary performance bottleneck in FOL is not the $O(\log N)$ algorithmic complexity of path-copying, but rather the memory-subsystem pressure caused by repeated intermediate allocations. In the naive QuickSort case, each swap generates multiple intermediate HAMT root nodes that are immediately discarded. The transient-builder protocol addresses this by allowing a sequence of mutations to occur within a single, thread-confined allocation window. This effectively "fuses" multiple functional updates into a single persistent result, bringing the cost of bulk updates much closer to the cost of mutable array manipulation. For the BFS workload, the 5.8× overhead of the transient version is particularly revealing: it suggests that once allocation costs are managed, the overhead of HAMT-based indexing is roughly five times that of a native hash table—a ratio that matches the depth of the 32-way trie branching.

## 4.4 Feature Benchmarks

We evaluated FOL across three benchmarks exercising specific language features: structural diff, hierarchical speculative execution, and observable slots. Table 6 summarizes performance and memory ratios.

**Table 6: Feature Benchmark Summary**

| Benchmark | Perf Ratio | Mem Ratio |
|---|---|---|
| Structural Diff | 113.7× | 55.3× |
| Hierarchical Speculative Execution | 23.4× | 9.0× |
| Observable Slots | 6.69× | 6.32× |

**Structural Diff** (500,000 iterations, 5-slot `<metric>` object): Each iteration makes six `assoc` calls—one `_changes` reset followed by five slot updates. Each call dispatches through the `fol.core:assoc` generic function, invokes the `<diffable>` `:around` method (reading the pre-update value and the post-update result), and when a change is detected triggers a *recursive* `assoc` to increment `_changes`—three HAMT path copies per changed slot. The 113.7× time overhead is compounding: CLOS dispatch, `:around` trampoline, HAMT path copy, and recursive re-entry all multiply. The 55.3× memory ratio reflects six HAMT copies per iteration plus recursive increments, versus a single struct allocation in the CL baseline (≈3,370 bytes vs. 61 bytes per iteration).

The memory overhead is *inherent* to the design: each `assoc` call must produce a fresh immutable snapshot. Production code replacing the six sequential `assoc` calls with a single `update-slots` batch would eliminate five of the six intermediate allocations; the benchmark uses sequential calls to isolate per-call overhead.

**Hierarchical Speculative Execution** (1,000,000 deposits through a three-level `:around` chain): The 23.4× overhead matches the single-guard case, confirming that the dominant cost is the HAMT path copy in the persistent-object update, not the method-combination chain depth. SBCL's PCL compiles the effective method for the three `:around` layers once and caches it; the chain adds no additional dispatch cost. The 9.0× memory ratio reflects one HAMT path copy per deposit (≈412 bytes vs. 46 bytes).

**Observable Slots** (1,000,000 sensor-update iterations): The 6.69× overhead reflects the cost of producing two persistent maps per iteration—the updated sensor state and the change event record. The 6.32× memory overhead stems from two HAMT allocations per update rather than the shallow struct copy of the CL baseline.

The feature benchmarks show overhead ratios from 6.69× to 113.7×, scaling with the number of `assoc` calls per logical operation. These ratios represent the fundamental cost of FOL's persistent object model; for workloads that require structural integrity, audit history, or safe rollback, the overhead is a consequence of correctness-by-construction rather than incidental implementation cost. Syntactic abstraction overhead is minimized by the compiler, which emits direct `defun` calls for internal function definitions. Remaining non-object overhead stems from closure allocation and runtime dispatch checks for late-bound or external symbols (where a call must support fallback to collection access).

## 4.5 Logic Simulation (LSim)

LSim is a digital logic simulator that exercises persistent state management at scale. We evaluated four configurations of increasing complexity: 8bit-100 (32-gate single 8-bit register, 100 time-steps),

32bit-300 (128-gate single 32-bit register, 300 time-steps), 8x32-900 (1024-gate pipeline of eight 32-bit registers, 900 time-steps), and 32x32-3000 (a 4096-gate pipeline of thirty-two cascaded 32-bit registers with 3000 time steps). Table 7 presents the results.

**Table 7: LSim Scaling Results**

| Config | CL | FOL | Ratio |
|---|---|---|---|
| 8bit-100 | 3 ms | 22 ms | 7.0× |
| 32bit-300 | 14 ms | 170 ms | 12.1× |
| 8x32-900 | 1.02 s | 4.26 s | 4.2× |
| 32x32-3000 | 48.3 s | 61.3 s | 1.27× |

The performance ratio narrows from 7.0× to 1.27× as circuit complexity grows. This trend reflects amortization rather than elimination of overhead: the absolute cost of persistent allocation remains, but it becomes a smaller fraction of total runtime as the simulation computation grows. At the largest scale (32x32-3000, ~4096 NAND gates, 3000 time steps), simulation work dominates and the persistent data structure overhead accounts for only 27% of total execution time.

The LSim simulation is implemented using an event-driven model where each NAND gate is represented as a persistent FOL object. Signal propagation triggers a cascade of `assoc` calls as gate outputs transition between logic states. In a traditional mutable implementation, each transition overwrites the gate's state, losing the history of the signal's propagation unless a global snapshot is explicitly taken. The CL baseline retains full per-step history by deep-copying the circuit state at each time step—a realistic requirement for event-replay and post-hoc analysis. This $O(N)$ copy is the natural mutable analog of FOL's persistent update. FOL's structural sharing allocates only $O(\log n)$ memory for the changed branches. GC pressure benchmarks for the 32x32-3000 circuit show that FOL allocates only 0.59× the bytes of the CL baseline and triggers 0.59× as many GC invocations. For workloads that do not require history, an in-place mutable CL implementation would allocate far less than either version; the 0.59× advantage is specific to the history-preserving comparison. The persistent model essentially provides a "free" undo-log and history mechanism that would otherwise require manual, high-overhead management in a mutable system.

## 4.6 Persistent Object Micro-Benchmarks

To isolate the cost of FOL's persistent metaclass from higher-level transpiler overhead, we measured construction, read, and functional update costs for persistent objects versus standard mutable CLOS objects across a range of slot counts. **Construction overhead** is modest (1.67–1.72× memory). **Native slot reads** are efficient (1.4–2.1× mutable CLOS), while overflow reads cost 3–5× due to MOP interception. **Functional updates** are the primary cost (33–447× slower than `setf`), amortized by batching. Table 9 evaluates the threshold selection: $T = 8$ minimizes update cost above $N \approx 28$ slots while keeping the common case on the native path.

The preceding construction figures compare FOL against mutable CLOS. Measured against a lean `defstruct` baseline—the representation a Clojure `defrecord` approximates—two distinct costs emerge. **Retained size**: each FOL object carries two hidden overhead slots (`%persistent-vector`, `%metadata`) on top of user slots, producing objects 2.5× larger than a struct for 1–2-slot classes (the statistical median) and 1.4× at 8 slots. Both the user slots and the two overhead slots are stored in a CLOS slot-vector—a separate heap allocation—giving each object a two-allocation footprint vs. a struct's one. **Construction GC pressure**: the bytes *consed* per allocation—the true GC pressure signal—are 8–15× higher than an equivalent CLOS construction: a 2-slot FOL object consumes ≈416 B/obj during construction vs. 48 B/obj for plain CLOS and 32 B/obj for a struct. The gap between construction cost (288–2,685 B/obj) and retained size (80–112 B/obj) shows that 80–95% of construction bytes are transient garbage from `initialize-instance` MOP machinery—slot iteration, initarg keyword dispatch, and internal CLOS caching. This is a secondary explanation for the high allocation ratios in the adversarial benchmarks (Section 4.3): each `assoc` call triggers not just a HAMT path copy but a full `initialize-instance` dispatch. It also explains why the transient-builder protocol reduces allocation by 97%: transients bypass `initialize-instance` entirely during the mutation window, writing directly into the mutable HAMT without object construction overhead.

A separate micro-benchmark evaluated the five candidate thresholds by directly simulating hybrid copy at each $T$ across object sizes $N \in \{8, 12, 16, 24, 32, 48\}$ (Table 9). Persistent vector batch trie construction costs ≈ 106 ns base plus 5–7 ns per element—an order of magnitude less per element than `slot-value` dispatch (≈ 27 ns/slot). For objects at or below the threshold (all-native path), all candidates are equivalent. The crossover occurs near $N \approx 28$: above this size, $T = 8$ minimizes update cost. Objects in the 9–24 slot range incur a modest penalty at $T = 8$ (0.12–0.17 $\mu$s, roughly 15–21%) from applying the vector base cost to small overflows. However, empirical OO class-size studies [8, 18] report the 90th percentile at ≈10–12 fields (median 2), placing $\lesssim$ 10% of classes in this penalty zone while ~80% remain all-native. Note that the simulated costs in Table 9 represent raw copy work; the actual `update-slots` costs (Table 8) are 1.6–3× higher due to MOP slot iteration, `slot-boundp` checks, and keyword-to-slot-name lookup overhead.

**Table 8: Actual slot update overhead (µs/op) vs. native CLOS**

| Slots | Persistent | Mutable | Ratio | Notes |
|---|---|---|---|---|
| 2 | 1.14 $\mu$s | 34.6 ns | 33× | 2 native slots |
| 8 | 2.34 $\mu$s | 41.6 ns | 56× | 8 native slots |
| 32 | 6.85 $\mu$s | 28.5 ns | 240× | all-native (T=32 config) |
| 48 | 8.50 $\mu$s | 41.6 ns | 204×[a] | 32N + 16-slot trie |
| 128 | 14.41 $\mu$s | 32.2 ns | 447× | 32N + 96-slot trie |

[a] Ratio drops vs. 32 slots because the mutable CLOS baseline is 46% slower at 48 slots (discontiguous memory layout), not because persistent overhead decreases.

## 4.7 Threats to Validity

Several threats to validity should be considered. **Single implementation**: All measurements use SBCL 2.6.0 on a single platform (AMD Ryzen 9, Windows 11); results may differ on other CL implementations or architectures. **Benchmark coverage**: Our evaluation includes both *feature benchmarks* (demonstrating the architectural synergy of persistence and dispatch) and *adversarial*

**Table 9: Modeled update cost ($\mu$s/op) at candidate thresholds $T$**

| N | T=8 | T=12 | T=16 | T=24 | T=32 | T=8 strategy |
|---|---|---|---|---|---|---|
| 8 | **0.42** | 0.42 | 0.43 | 0.42 | 0.42 | all-native |
| 12 | 0.75 | **0.62** | 0.62 | 0.61 | 0.60 | 8N + 4V |
| 16 | 0.94 | 0.91 | 0.83 | 0.78 | **0.77** | 8N + 8V |
| 24 | **1.15** | 1.23 | 1.28 | **1.15** | **1.15** | 8N + 16V |
| 32 | **1.39** | 1.47 | 1.51 | 1.58 | 1.52 | 8N + 24V |
| 48 | **1.95** | 1.99 | 2.04 | 2.12 | 2.22 | 8N + 40V |

Bold = minimum in row. "$k$N + $j$V" = copy $k$ native slots then batch-build $j$-element overflow vector. All-native = copy $N$ slots directly. Simulated copy costs only; actual `update-slots` is 1.6–3× higher due to MOP iteration overhead (see Table 8).

*workloads* (quantifying the performance penalty of naive persistent algorithms). While this provides a comprehensive view of FOL's performance envelope, the feature benchmarks naturally select for patterns where FOL excels, and the adversarial benchmarks isolate worst-case overheads that may be mitigated in production through idiomatic persistent algorithms or the transient-builder protocol. Benchmarks report means of multiple runs to amortize warmup and JIT effects. As a single-author project, FOL has not yet undergone independent review; the source is available for external verification.

FOL is transpiled to Common Lisp, introducing a threat specific to dispatch performance: predicate specializers (Levels 2–3) cannot be cached by class signature and require runtime evaluation on each invocation, so reported dispatch overhead compounds FOL's routing cost with SBCL's PCL cache-miss penalty. A native compiler could replace the $O(N)$ scan with an ordered type-check sequence; the dispatch ratios reported here are therefore upper bounds on inherent routing cost.

No static analysis exists—type errors and unreachable clauses are detected at runtime only; some type-dependent code may require runtime resolution, degrading transpiled performance. The compiler has not been extended to warn about shadowed clauses; a future version could detect shadowing statically.

The specificity calculus struggles with compound predicates; conjunctions such as `(and (<number>?) (even?))` fall to Level 3 even if they logically narrow a Level 2 type. This highlights the inherent difficulty of providing a sound, decidable subtyping relationship over arbitrary dynamic logic. A recursive extension—$\mathcal{L}(\text{and } p\ q) = \max(\mathcal{L}(p), \mathcal{L}(q))$—would lift compound type-predicate conjunctions but cannot order disjoint Level 3 predicates whose containment is undecidable. Definition-order tie-breaking, while a *modularity hazard* where load order determines behavior, aligns with Lisp's interactive, last-definition-wins workflow. A strict approach signaling ambiguity errors would improve modularity but disrupt the development ergonomics typical of the ecosystem.

The dominant performance limitation remains the functional update cost (Section 4.3). Transient builders (Section 3.1) address initialization-heavy operations by batching slot writes within a delimited scope, reducing the cost to mutable-equivalent construction; the residual overhead for single-slot mutation loops is inherent to the persistent model and requires idiomatic persistent algorithms or acceptance of mutable structures for those paths. Class redefinition does not migrate existing instances (Section 3.3); old objects retain their original slot data, with new slots resolved lazily via initforms.

Planned extensions include **dispatch stratification** (implementing the two-strata model described in Section 3.2 using decision trees and fast-path caches for Levels 0–2), **metadata-based type markers** to accelerate predicate evaluation, **abstract and sealed classes** to enable better static deviance checking, **parallel collections** that leverage structural sharing for fork-join parallelism, **channel-based concurrency** (CSP-style go-blocks) integrated with the sequence protocol, a **native compiler** targeting LLVM or machine code directly, and a **VSCode extension** for integrated editing and debugging.

## 5 Related Work

**Clojure** [14] pioneered persistent data structures in Lisp. FOL adopts its sequence and transducer abstractions but trades protocol-based dispatch for full CLOS/MOP. **Common Lisp**'s CLOS [4] and MOP [17] provide the foundation; while libraries like FSet [6] provide persistence, they lack MOP-integrated metadata management. **Elephant** [12] explored metaclass-based persistence but focused on disk-backed storage rather than structural sharing. **Dylan** [1] inspired FOL's naming conventions and its pure OO focus. **Predicate dispatch** builds on Cecil [7, 11], whose subsumption-based specificity is more general but undecidable; FOL's category-based ordered dispatch ensures predictability by inspection. **Racket** [13] offers a distinct synthesis of records and classes; FOL differs by unifying them such that every object *identity* is both a CLOS instance and a persistent record, enabling metaclass-level interception of updates that Racket's separate struct/class hierarchies cannot easily coordinate.

FOL's functional update syntax (`update-slots`) mirrors record updates in **F#** and **Elm** [9]. While **Julia** [3] uses multiple dispatch, it lacks predicate specializers and method combinations. **Coalton** [19] and **Typed Racket** [20] focus on static safety, whereas FOL prioritizes persistence. FOL's lazy migration (Section 3.3) shares goals with dynamic evolution in **Self** [21] and **Newspeak** [5], but applies them to immutable state: old snapshots remain valid naturally as migration occurs only during functional updates.

## 6 Conclusion

FOL demonstrates a successful synthesis of persistent data structures and CLOS, yielding architectural patterns that are difficult to achieve in either paradigm in isolation. Our evaluation confirms that while persistent allocation introduces significant micro-overhead (up to 113.7×), this cost is effectively amortized in non-trivial workloads, reaching near-parity (1.27×) in large-scale simulations. Crucially, we find that the efficiency of structural sharing actually inverts the naive expectation of GC pressure at scale: FOL allocates only 0.59× the bytes of a deep-copying mutable equivalent, indicating that persistence is a modular engineering cost that can be managed and even leveraged through principled MOP-level design. This work shows that functional object orientation is a viable and expressive choice for modern, state-heavy systems. The complete implementation is available at: https://github.com/frankadrian314159/fol

# References


[1] Apple Computer, Inc. 1992. *Dylan - An Object-Oriented Dynamic Programming Language*. Apple Computer, Inc.

[2] Erik Barendsen and Sjaak Smetsers. 1993. Uniqueness Typing for Functional Languages. In *Proceedings of the 5th International Workshop on Implementation of Functional Languages*. 1–25.

[3] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Rev.* 59, 1 (2017), 65–98.

[4] Daniel G Bobrow, Linda G DeMichiel, Richard P Gabriel, Sonya E Keene, Gregor Kiczales, and David A Moon. 1988. Common Lisp Object System Specification. *SIGPLAN Notices* 23, SI (1988), 1–142.

[5] Gilad Bracha, Peter von der Ahé, Vassili Bykov, Yaron Kashai, William Maddox, and David Ungar. 2010. The Newspeak programming language. In *Proceedings of the 19th symposium on Dynamic languages*.

[6] Scott L Burke. 2007. FSet: A functional set-theoretic collections library. https://common-lisp.net/project/fset/

[7] Craig Chambers. 1993. The Cecil Language: Design and Performance. In *Proceedings of the Conference on Object-Oriented Programming Systems, Languages, and Applications*. 243–257.

[8] Giulio Concas, Michele Marchesi, Sandro Pinna, and Nicola Serra. 2007. Power Laws in a Large Object-Oriented Software System. *IEEE Transactions on Software Engineering* 33, 10 (2007), 687–708.

[9] Evan Czaplicki. 2012. *Elm: Concurrent FRP for Functional GUIs*. Master's thesis. Harvard University.

[10] James R Driscoll, Neil Sarnak, Daniel D Sleator, and Robert E Tarjan. 1989. Making Data Structures Persistent. *J. Comput. System Sci.* 38, 1 (1989), 86–124.

[11] Michael D Ernst, Craig Kaplan, and Craig Chambers. 1998. Predicate Dispatching: A Unified Theory of Dispatch. In *ECOOP'98—Object-Oriented Programming*. Springer, 186–211.

[12] Ian Eslick and Ben Hyde. 2008. Elephant: A Persistent Object Database for Common Lisp. https://common-lisp.net/project/elephant/

[13] Matthew Flatt et al. 2015. The Racket Manifesto. In *20 Years of the Past and 20 Years of the Future (Snapshots), Dagstuhl Seminar 15451*.

[14] Rich Hickey. 2008. The Clojure programming language. In *Proceedings of the 2008 symposium on Dynamic languages*. 1.

[15] Rich Hickey. 2009. Are We There Yet? JVM Languages Summit Keynote. https://github.com/matthiasn/talk-transcripts/blob/master/Hickey_Rich/AreWeThereYet.md

[16] Rich Hickey. 2014. Transducers. Clojure Blog. https://clojure.org/reference/transducers

[17] Gregor Kiczales, Jim Des Rivieres, and Daniel Gureasko Bobrow. 1991. *The Art of the Metaobject Protocol*. MIT press.

[18] Radu Muschevici, Alex Potanin, Ewan Tempero, and James Noble. 2008. The Secret Life of Objects: Measures of Object-Oriented Programs. In *Proceedings of the 9th International Conference on Objects, Components, Models and Patterns*. Springer, 52–71.

[19] Robert Smith and Cole Bates. 2023. Coalton: Efficient, Statically Typed Functional Programming on Common Lisp. In *Proceedings of the 16th European Lisp Symposium*.

[20] Sam Tobin-Hochstadt and Matthias Felleisen. 2008. The Design and Implementation of Typed Scheme. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 395–406.

[21] David Ungar and Randall B Smith. 1987. Self: The power of simplicity. In *ACM SIGPLAN Notices*, Vol. 22. ACM, 227–242.

[22] Philip Wadler. 1990. Linear Types Can Change the World!. In *Programming Concepts and Methods*.